TECHNICAL UNIVERSITY OF MOLDOVA
Faculty of Computers, Informatics, and Microelectronics
Department of Software Engineering
Software Engineering Study Programme

LABORATORY REPORT

OPERATING SYSTEMS LABORATORY 8

The Same Calls Inside Your Own OS

Done by: st. gr. FAF-241, Lucian-Adrian Gavril

Checked by: univ. assist., Reițman Patricia

Chișinău, 2026

TABLE OF CONTENTS

ABSTRACT 3

INTRODUCTION 4

1 PROCESSES AND EXEC 5

2 EXEC VARIANTS 8

3 TWO PIPES 10

4 QEMU OBSERVATIONS 12

5 OPTIONAL PRIME-FILTER CHAIN 14

CONCLUSIONS 15

REFERENCES 16

APPENDICES 17

APPENDIX A – Program source 17

A.1 user/run.c 17

A.2 user/pingpong.c 18

A.3 user/primes.c 20

ABSTRACT

This laboratory repeats process creation, program replacement and waiting inside xv6, then implements a one-byte request and reply using two pipes. The report compares the xv6 and Linux interfaces, traces the kernel boundary and explains descriptor ownership. A clean build and QEMU boot demonstrate echo, the successful and failed exec variants, ordered ping-pong output and an optional process chain filtering primes from 2 through 35.

Keywords: processes, system calls, file descriptors, pipes, linux, xv6

INTRODUCTION

The required experiments follow the laboratory handout[1]; the process and descriptor model follows the course lecture[2]. The supplied source is reproduced in Appendix A. A successful execution is evidence for that captured environment, while the concept figures explain the general mechanism.

The UNIX process interface lets a shell compose small programs without changing their implementations. Process calls control which program runs and when its completion is collected; file descriptors select where the program reads and writes. These mechanisms explain redirection, pipelines and communication between processes. The required `run` program demonstrates `fork`, `exec` and `wait`; user/pingpong.c demonstrates a one-byte exchange over two pipes. The QEMU observations below record the behavior of these programs in xv6.

Figure 1.1 distinguishes a new process from a replacement program. The amber interval represents an exited child retained until its parent reaps it; the child executes no instructions in that interval.

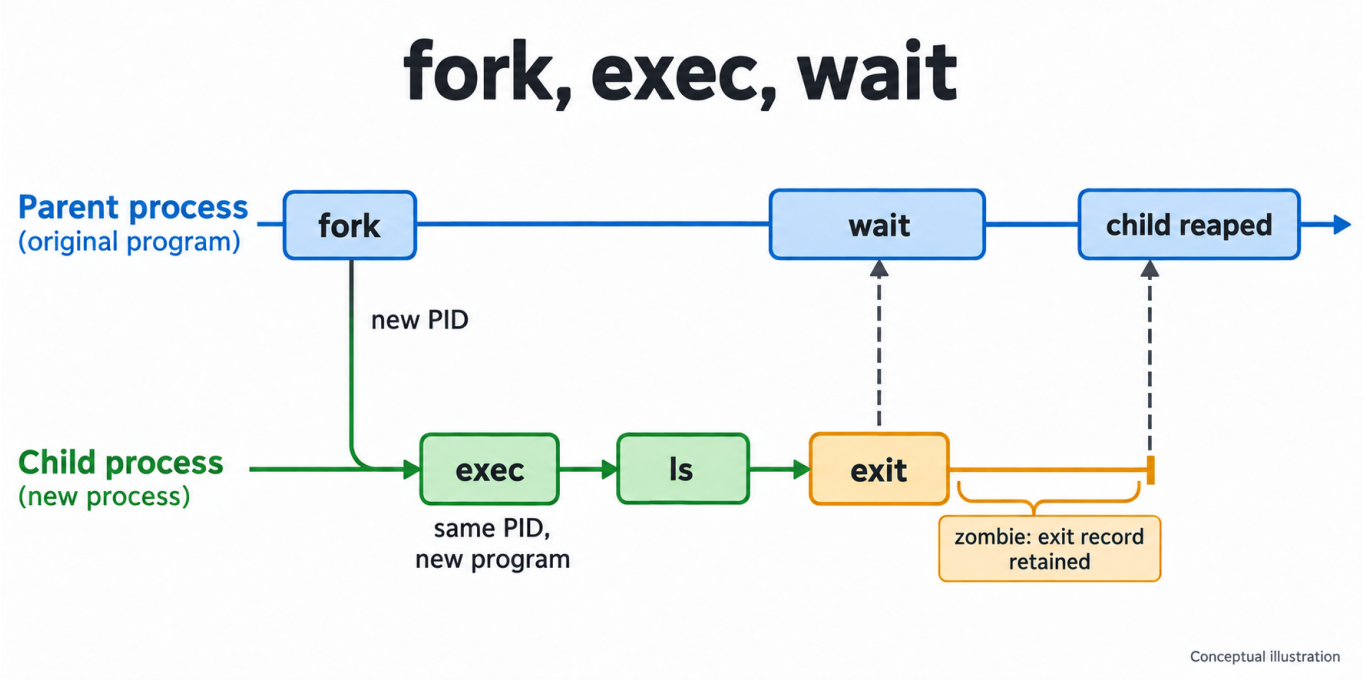


Figure 1.1 – Process lifetime and the retained exit record. ImageGen concept illustration, 23 October 2026; scheduling is schematic.

The xv6 process interface is small enough to trace through its source. `user/usys.pl` generates wrappers that place the system-call number in register `a7` and execute `ecall`. The trap path enters `usertrap`; `kernel/syscall.c` dispatches the request. Process creation and waiting are implemented in `kernel/proc.c`; `kernel/exec.c` constructs the replacement user image[3].

In `user/run.c`, the child passes an argument vector to `exec("echo", args)`. The vector contains the executable name, four argument words and a null terminator. The loaded `echo` program prints the argument words. The parent's later wait obtains completion information rather than the child's text output.

1. xv6 and Linux use the same basic sequence: `fork` creates a child process, `exec` replaces the child's program image, and the parent calls a wait operation to collect the child's completion. The APIs differ. xv6 provides `exec(path, argv)` and `wait(&status)`; Linux commonly uses `execlp` and `waitpid`. `execlp` searches `PATH`, while xv6 `exec` is passed a path directly. Linux's wait status is encoded and normally decoded with macros such as `WEXITSTATUS`; this xv6 implementation copies the child's integer exit value to `status`;
2. The child process prints `child pid` before the call to `exec`. If `exec` succeeds, that child becomes the `echo` program. The `echo` program prints `hello from the child`. The parent remains in `run` and prints the child's PID and status only after `wait` returns;

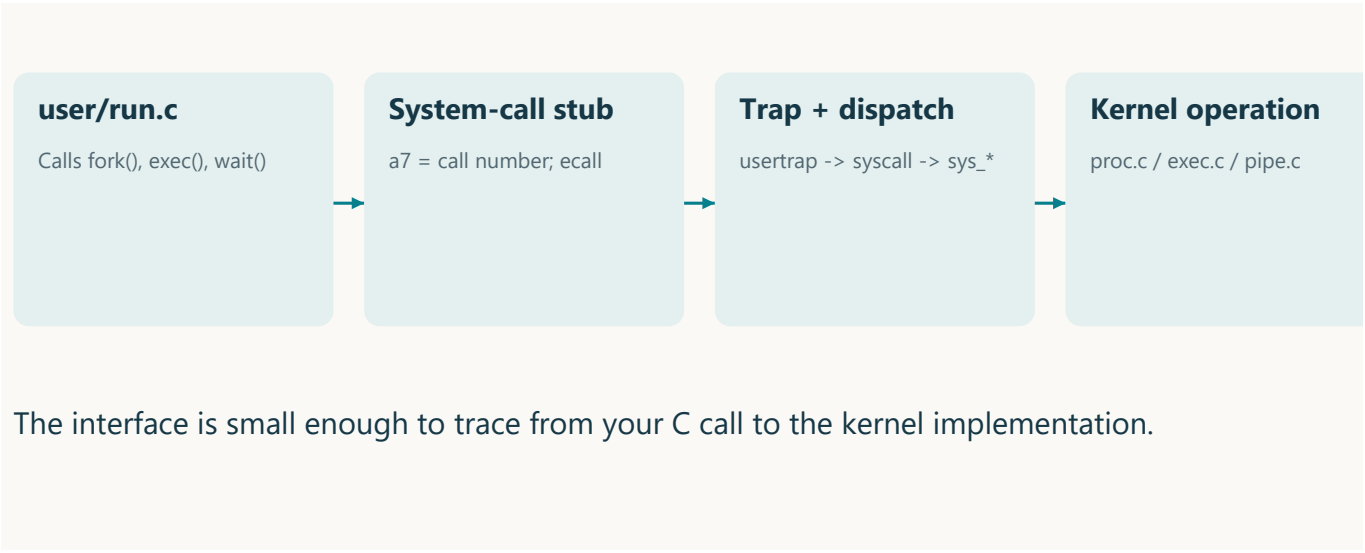


Figure 1.2 – The user-call stub enters the kernel through ecall; the dispatcher selects the requested kernel operation.

3. The missing-program variant prints `exec failed` and explicitly calls `exit(1)`, so xv6's `wait(&status)` reports status 1. In Lab 7's Linux `run.c`, the `execlp` failure branch instead calls `exit(127)`, and the parent decodes 127 with `WEXITSTATUS(status)`. Both values are chosen by the child program's error branch; a failed `exec` call does not itself choose an exit status. Shells also commonly use 127 for a command they cannot find;

```
$ riscv64-unknown-elf-gcc --version
riscv64-unknown-elf-gcc (13.2.0-11ubuntu1+12) 13.2.0
Copyright (C) 2023 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

$ qemu-system-riscv64 --version
QEMU emulator version 8.2.2 (Debian 1:8.2.2+ds-0ubuntu1.18)
Copyright (c) 2003-2023 Fabrice Bellard and the QEMU Project developers
$ grep -E '_run|_pingpong|_primes' Makefile
        $U/_run\
        $U/_runls\
        $U/_runmissing\
        $U/_pingpong\
        $U/_primes\
$ make -s clean
$ make -s fs.img kernel/kernel
riscv64-unknown-elf-ld: warning: user/_forktest has a LOAD segment with RWX permissions
nmeta 47 (boot, super, log blocks 31, inode blocks 13, bitmap blocks 1) blocks 1953 total 2000
balloc: first 1172 blocks have been allocated
balloc: write bitmap block at sector 46
riscv64-unknown-elf-ld: warning: kernel/kernel has a LOAD segment with RWX permissions
Clean xv6 build exited with status 0.
$ make -s qemu

xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$
```

Figure 1.3 – Clean xv6 build, required UPROGS entries, tool versions and successful QEMU boot. Live ttyd terminal capture, 19 October 2026. Only blank margins are cropped in this report; program output is unchanged.

```
Lab 8 | Canonical run: child becomes echo
Live execution | 2026-10-19 16:13:21


$ run
parent pid: 3
child pid: 4
hello from the child
child 4 ended with status 0
$
```

Figure 1.4 – The canonical run program starts an echo child and collects exit status 0. Live ttyd terminal capture, 19 October 2026. Only blank margins are cropped in this report; program output is unchanged.

2 EXEC VARIANTS

Changing only the argument array and target tests a successful `ls` replacement and a failed `nosuchprog` replacement. The saved `user/run.c` remains the required echo example. In the missing-program case, `exec` returns an error, so the child reaches the diagnostic and `exit(1)`. For `ls`, successful `exec` never returns to `run`'s child branch.

```
Lab 8 | Clean xv6 build, registrations and QEMU boot
Live execution | 2026-10-19 16:12:35

Lab 8 | Successful ls exec variant
Live execution | 2026-10-19 16:13:32


$ runls
parent pid: 5
child pid: 6
.              1 1 1024
..             1 1 1024
README         2 2 2473
cat            2 3 36512
echo           2 4 35360
forktest       2 5 17080
grep           2 6 39944
init           2 7 35808
kill           2 8 35280
ln             2 9 35096
ls             2 10 38624
mkdir          2 11 35344
rm             2 12 35328
sh             2 13 58344
sleep          2 14 35080
args           2 15 35080
stressfs       2 16 36208
usertests      2 17 186256
grind          2 18 51744
wc             2 19 37488
zombie         2 20 34680
```

Figure 2.1 – The overlay-only runls command starts parent 5 and child 6, which executes ls; the listing begins here. Live ttyd terminal capture, 19 October 2026. Only blank margins are cropped in this report; program output is unchanged.

```
..              1 1 1024
README          2 2 2473
cat             2 3 36512
echo            2 4 35360
forktest        2 5 17080
grep            2 6 39944
init            2 7 35808
kill            2 8 35280
ln              2 9 35096
ls              2 10 38624
mkdir           2 11 35344
rm              2 12 35328
sh              2 13 58344
sleep           2 14 35080
args            2 15 35080
stressfs        2 16 36208
usertests       2 17 186256
grind           2 18 51744
wc              2 19 37488
zombie          2 20 34680
logstress       2 21 37280
run             2 22 35608
runls           2 23 35576
runmissing      2 24 35592
pingpong        2 25 36696
primes          2 26 38024
forphan         2 27 36104
dorphan         2 28 35552
console         3 29 0
child 6 ended with status 0
$
```

Figure 2.2 – Continuation of the same ls variant: the listing finishes and the parent collects child 6 with status 0. Live ttyd terminal capture, 19 October 2026. Only blank margins are cropped in this report; program output is unchanged.

```
Lab 8 | Missing program: exec returns and child exits 1
Live execution | 2026-10-19 16:14:07


$ runmissing
parent pid: 7
child pid: 8
exec failed
child 8 ended with status 1
$
```

Figure 2.3 – The missing-program variant reaches exec failed and the explicit exit(1) branch. Live ttyd terminal capture, 19 October 2026. Only blank margins are cropped in this report; program output is unchanged.

Figure 3.1 shows why closing unused endpoints is part of the algorithm. The two-direction xv6 exchange applies this rule independently to each pipe.

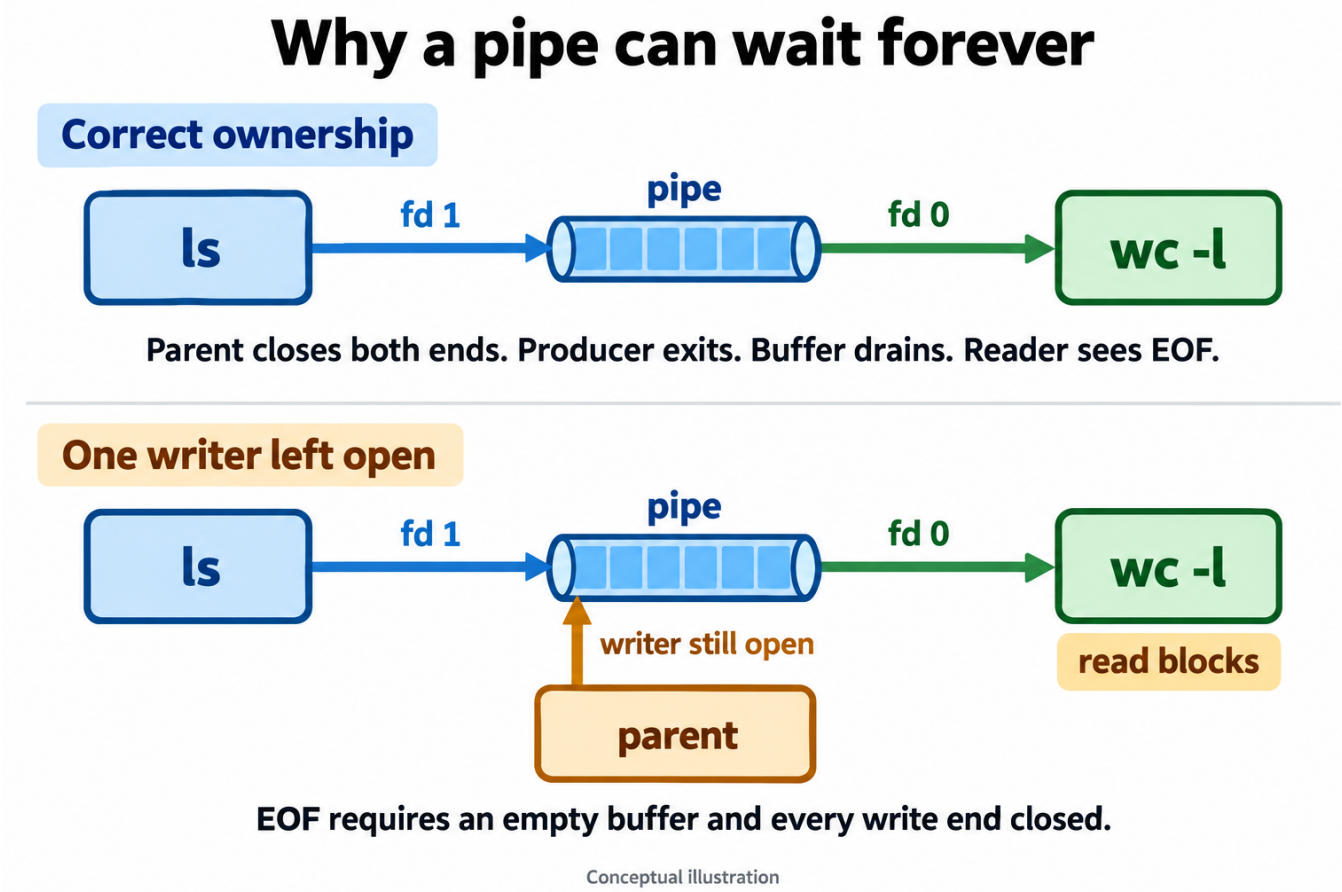


Figure 3.1 – Writer ownership determines end-of-file. ImageGen concept illustration, 23 October 2026; not measured output.

The pipes are created before the fork so both processes inherit their endpoints. `parent_to_child` carries the request; `child_to_parent` carries the reply. The parent initializes a one-byte token. Every read and write checks that one byte was transferred. The two streams give each direction a separate reader, avoiding a race in which a process could read its own message from one shared pipe.

The parent writes the ping byte to `parent_to_child[1]`; the child reads it from `parent_to_child[0]`. The child writes the pong byte to `child_to_parent[1]`; the parent reads it from `child_to_parent[0]`. xv6 pipes are one-way streams, so the two directions use separate pipes. With one shared pipe, both processes would read and write through the same stream, and a reader could consume the byte it had just written.

The parent blocks in `read(child_to_parent[0], ...)` until the child writes its reply. `wait(0)` comes after the reply and collects the child's exit; it is not the call that waits for the pong byte. Each process closes the ends it does not use. That makes the ownership of each direction explicit and allows EOF to reach

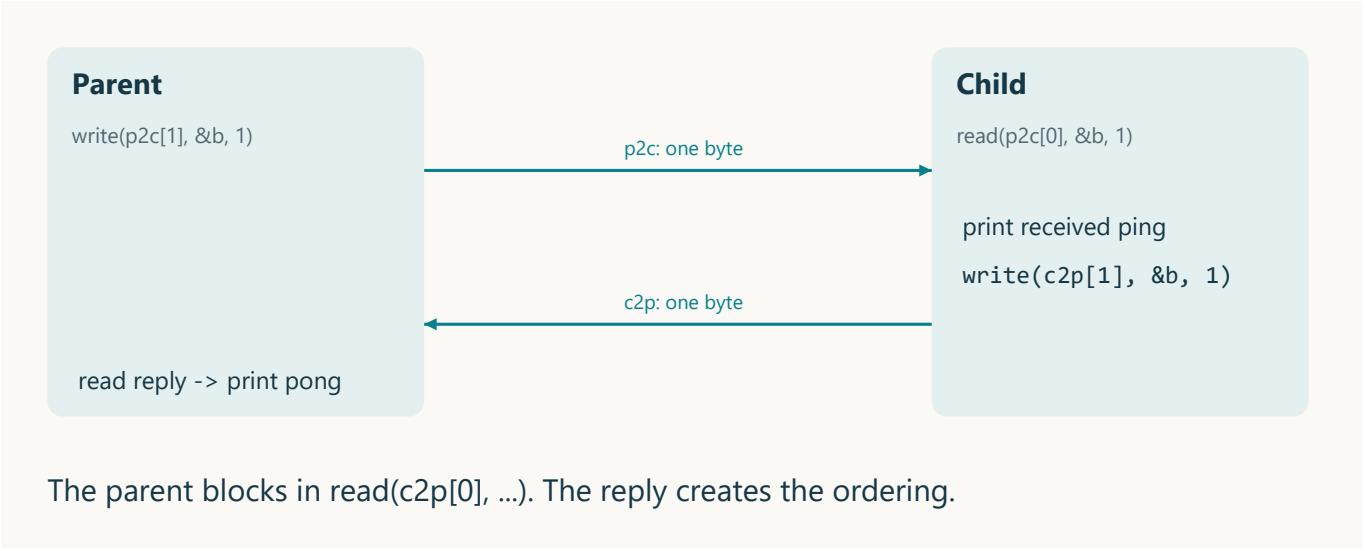


Figure 3.2 – A request byte travels parent to child; its reply travels child to parent on a separate pipe.

a reader when the remaining writers close.

```
Lab 8 | Two pipes: ping before pong
Live execution | 2026-10-19 16:14:18


$ pingpong
10: received ping
9: received pong
$
```

Figure 3.3 – Actual xv6 execution: the child prints received ping before the parent prints received pong. Live ttyd terminal capture, 19 October 2026. Only blank margins are cropped in this report; program output is unchanged.

4 QEMU OBSERVATIONS

The programs build cleanly with RISC-V GCC 13.2.0 and boot under QEMU 8.2.2. The captured console output and invocation are in the fresh capture transcript. The successful run uses a disposable Lab 2 overlay and the Makefile's default three CPUs.

Observed in that QEMU session:

```
$ run

parent pid: 3

child pid: 4

hello from the child

child 4 ended with status 0


$ pingpong

10: received ping

9: received pong


$ primes

prime 2

prime 3

prime 5

prime 7

prime 11

prime 13

prime 17

prime 19

prime 23

prime 29

prime 31
```

For the successful `ls` variant, an overlay-only build printed the xv6 root-directory entries and the parent observed child status 0. For the missing-program variant, an overlay-only build printed `exec failed`, and the parent observed child status 1. Those two temporary executables were generated from `run.c` by

changing only its argument array and `exec` target; the submitted source remains the echo example. Process IDs depend on the boot and are recorded here only for this run.

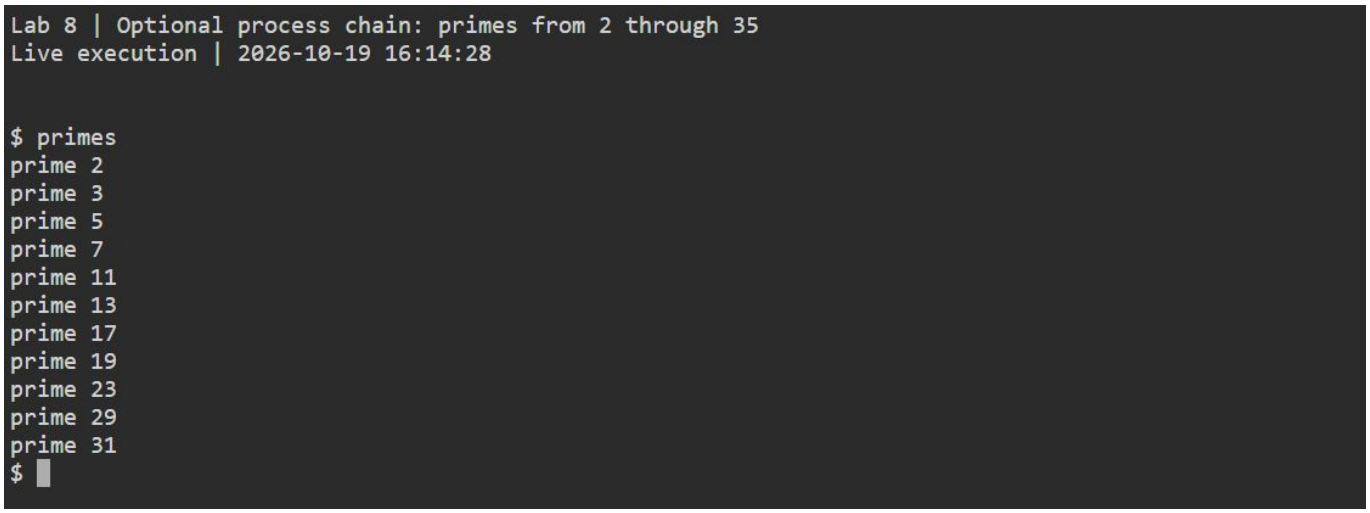


Figure 4.1 – The optional process chain prints exactly the primes from 2 through 35. Live ttyd terminal capture, 19 October 2026. Only blank margins are cropped in this report; program output is unchanged.

5 OPTIONAL PRIME-FILTER CHAIN

The producer sends the integers 2 through 35. Each stage reads its first surviving number as a prime, prints it and passes only numbers not divisible by that prime to a new child stage. Earlier stages have already removed multiples of smaller primes. Every stage closes its output when its input ends, allowing EOF to propagate, and waits for the child it created. The observed eleven outputs are listed in the QEMU results.

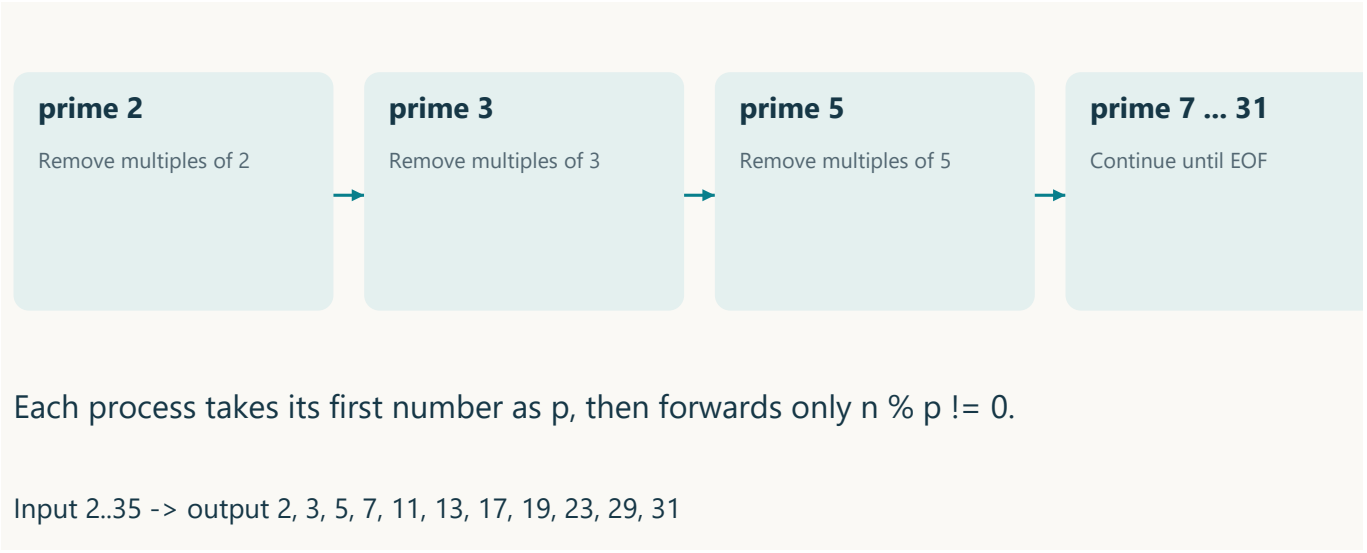


Figure 5.1 – Each process removes multiples of one prime and forwards the remaining integers.

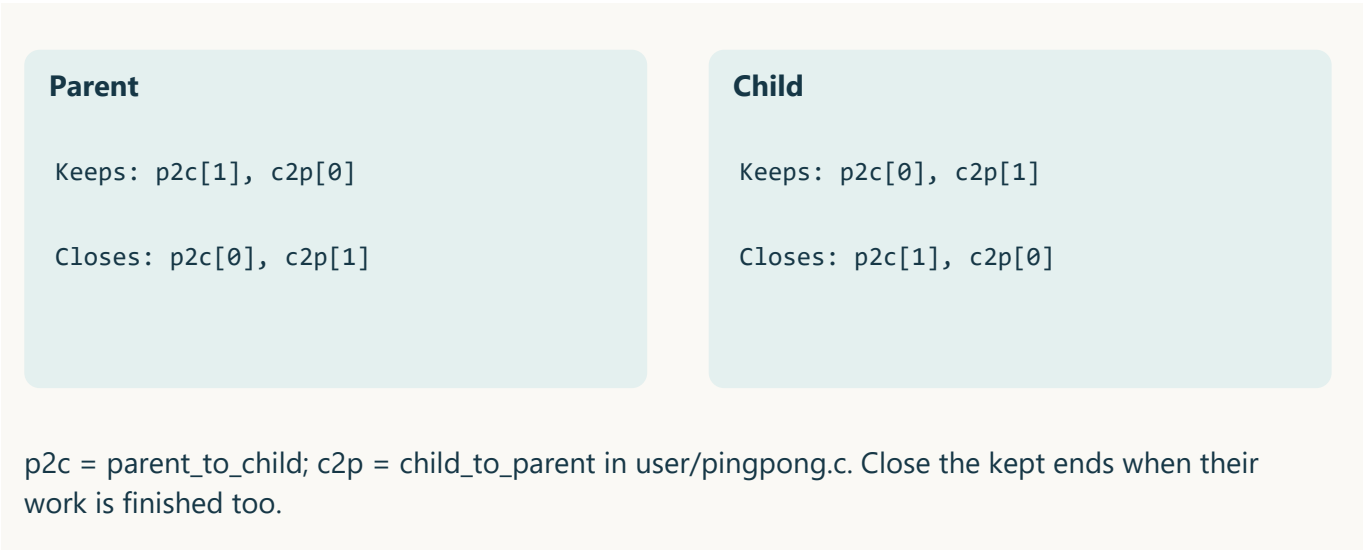


Figure 5.2 – After fork, each process closes the endpoint copies it does not use, then closes its used ends after the exchange.

CONCLUSIONS

The xv6 run demonstrates the same process lifecycle as the Linux example while exposing its kernel implementation. The echo and ls children exited successfully; the explicit failed-exec branch returned status 1. Two pipes made the ping and reply directions independent. Blocking read enforced ping-before-pong ordering, and the later wait reaped the child. The optional filter chain demonstrated the same ownership and EOF rules across multiple stages.

REFERENCES

1. Operating Systems, FCIM/FAF, UTM. *Laboratory 8 handout*, academic year 2026–2027. Accessed 19 October 2026. `https://else.fcim.utm.md/mod/resource/view.php?id=72240`.

2. Operating Systems. *Processes, Descriptors, Pipes*, Session 4. Accessed 19 October 2026. `https://else.fcim.utm.md/mod/resource/view.php?id=72236`.

3. MIT. *xv6-riscv source code*. Course source inspected 19 October 2026. Local course snapshot: `os/lab2`; `kernel/proc.c`, `kernel/exec.c`, `kernel/pipe.c` and `user/sh.c`.

APPENDICES

APPENDIX A – Program source

The following listings are the submitted programs. They accompany the observations in the main report.

A.1 user/run.c

```
#include "kernel/types.h"
#include "user/user.h"

int
main(void)
{
  int child_pid;
  int child_status;

  printf("parent pid: %d\n", getpid());

  child_pid = fork();
  if(child_pid < 0){
    printf("fork failed\n");
    exit(1);
  }

  if(child_pid == 0){
    char *args[] = {"echo", "hello", "from", "the", "child", 0};

    printf("child pid: %d\n", getpid());
    exec("echo", args);
    printf("exec failed\n");
    exit(1);
  }

  if(wait(&child_status) < 0){
    printf("wait failed\n");
    exit(1);
  }

  printf("child %d ended with status %d\n", child_pid, child_status);
```

```
  exit(0);
}
```

A.2 user/pingpong.c

```
#include "kernel/types.h"
#include "user/user.h"

int
main(void)
{
  int parent_to_child[2];
  int child_to_parent[2];
  int child_pid;
  int bytes;
  char token = 'x';

  if(pipe(parent_to_child) < 0){
    printf("pingpong: first pipe failed\n");
    exit(1);
  }
  if(pipe(child_to_parent) < 0){
    printf("pingpong: second pipe failed\n");
    close(parent_to_child[0]);
    close(parent_to_child[1]);
    exit(1);
  }

  child_pid = fork();
  if(child_pid < 0){
    printf("pingpong: fork failed\n");
    exit(1);
  }

  if(child_pid == 0){
    close(parent_to_child[1]);
    close(child_to_parent[0]);

    bytes = read(parent_to_child[0], &token, 1);
    if(bytes != 1){
      printf("pingpong: child read failed\n");
```

```
37        exit(1);
38      }
39      printf("%d: received ping\n", getpid());
40
41      bytes = write(child_to_parent[1], &token, 1);
42      if(bytes != 1){
43        printf("pingpong: child write failed\n");
44        exit(1);
45      }
46
47      close(parent_to_child[0]);
48      close(child_to_parent[1]);
49      exit(0);
50    }
51
52    close(parent_to_child[0]);
53    close(child_to_parent[1]);
54
55    bytes = write(parent_to_child[1], &token, 1);
56    if(bytes != 1){
57      printf("pingpong: parent write failed\n");
58      exit(1);
59    }
60    close(parent_to_child[1]);
61
62    bytes = read(child_to_parent[0], &token, 1);
63    if(bytes != 1){
64      printf("pingpong: parent read failed\n");
65      exit(1);
66    }
67    printf("%d: received pong\n", getpid());
68    close(child_to_parent[0]);
69
70    if(wait(0) < 0){
71      printf("pingpong: wait failed\n");
72      exit(1);
73    }
74    exit(0);
75  }
```

A.3 user/primes.c

```
#include "kernel/types.h"
#include "user/user.h"

static void filter(int input_fd) __attribute__((noreturn));

static void
filter(int input_fd)
{
  int prime;
  int bytes = sizeof(prime);
  int result = read(input_fd, &prime, bytes);
  int next_pipe[2];
  int child_pid;
  int candidate;

  if(result == 0){
    close(input_fd);
    exit(0);
  }
  if(result != bytes){
    printf("primes: failed to read a number\n");
    close(input_fd);
    exit(1);
  }

  printf("prime %d\n", prime);

  if(pipe(next_pipe) < 0){
    printf("primes: pipe failed\n");
    close(input_fd);
    exit(1);
  }

  child_pid = fork();
  if(child_pid < 0){
    printf("primes: fork failed\n");
    close(input_fd);
    close(next_pipe[0]);
    close(next_pipe[1]);
```

```
40      exit(1);
41    }
42
43    if(child_pid == 0){
44      close(input_fd);
45      close(next_pipe[1]);
46      filter(next_pipe[0]);
47    }
48
49    close(next_pipe[0]);
50
51    while((result = read(input_fd, &candidate, bytes)) > 0){
52      if(result != bytes){
53        printf("primes: incomplete number\n");
54        close(input_fd);
55        close(next_pipe[1]);
56        wait(0);
57        exit(1);
58      }
59
60      if(candidate % prime != 0){
61        if(write(next_pipe[1], &candidate, bytes) != bytes){
62          printf("primes: write failed\n");
63          close(input_fd);
64          close(next_pipe[1]);
65          wait(0);
66          exit(1);
67        }
68      }
69    }
70
71    close(input_fd);
72    close(next_pipe[1]);
73    if(result < 0){
74      printf("primes: read failed\n");
75      wait(0);
76      exit(1);
77    }
78    if(wait(0) < 0){
79      printf("primes: wait failed\n");
```

```
80      exit(1);
81    }
82    exit(0);
83  }
84
85  int
86  main(void)
87  {
88    int input_pipe[2];
89    int number;
90    int child_pid;
91    int bytes = sizeof(number);
92
93    if(pipe(input_pipe) < 0){
94      printf("primes: input pipe failed\n");
95      exit(1);
96    }
97
98    child_pid = fork();
99    if(child_pid < 0){
100     printf("primes: producer fork failed\n");
101     close(input_pipe[0]);
102     close(input_pipe[1]);
103     exit(1);
104   }
105
106   if(child_pid == 0){
107     close(input_pipe[1]);
108     filter(input_pipe[0]);
109   }
110
111   close(input_pipe[0]);
112   for(number = 2; number <= 35; number++){
113     if(write(input_pipe[1], &number, bytes) != bytes){
114       printf("primes: input write failed\n");
115       close(input_pipe[1]);
116       wait(0);
117       exit(1);
118     }
119   }
```

```
120   close(input_pipe[1]);
121
122   if(wait(0) < 0){
123     printf("primes: producer wait failed\n");
124     exit(1);
125   }
126   exit(0);
127 }
```